Computer Graphics Assignment – Group 11

Members:

- Eunice Nyaboke – SCT211-0544/2022
- Victor Ireri – SCT211-0435/2023

Task 1: Frame Buffer Address Calculation

Problem:

Monitor 12″ × 14″, 120 pixels/inch, 4 bits/pixel, 1-byte words, start address 0. Find address of pixel `(x, y)`.

Solution:

- Width = 12 × 120 = **1440 pixels** → x ∈ [0, 1439]
- Height = 14 × 120 = **1680 pixels** → y ∈ [0, 1679]
- 4 bits/pixel = 0.5 bytes/pixel → 2 pixels per byte.

Final formula:

$$address = \lfloor y \times 1440 + x2 \rfloor \text{address} = \lfloor 2y \times 1440 + x \rfloor$$

Task 2: OpenGL Program – Translation (green/cream) + Rotation (55°)

Requirements Recap

- Original quadrilateral: A(0,4), B(3,4), C(4,0), D(0,0)
- Translate by (+2, +2) → A′(2,6), B′(5,6), C′(6,2), D′(2,2)
- **Translated figure** must have **green border** and **cream fill**.
- Rotate the **translated** figure by **55°** about origin.
- Display with labelled axes (ticks, numbers, 'X', 'Y').

Complete C++ Code (copy-paste into `main.cpp`)

```cpp
/*
 * Group 11 – Eunice Nyaboke & Victor Ireri
 * OpenGL: Translation (green/cream) + Rotation (55°) with labelled axes
 */

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif

#include <cmath>
#include <cstdio>
#include <cstring>

#define PI 3.14159265358979323846
```

```
float original[4][2] = {{0,4},{3,4},{4,0},{0,0}};
float translated[4][2], rotated[4][2];
const float tx = 2, ty = 2;
const float theta_deg = 55;
float theta_rad;

void computeTranslated() {
    for (int i = 0; i < 4; i++) {
        translated[i][0] = original[i][0] + tx;
        translated[i][1] = original[i][1] + ty;
    }
}

void computeRotated() {
    float c = cos(theta_rad), s = sin(theta_rad);
    for (int i = 0; i < 4; i++) {
        float x = translated[i][0], y = translated[i][1];
        rotated[i][0] = x * c - y * s;
        rotated[i][1] = x * s + y * c;
    }
}

void drawPolygon(float v[4][2], float fr, float fg, float fb, float br, float bg, float
bb, float lw=2.5) {
    glColor3f(fr, fg, fb);
    glBegin(GL_POLYGON);
    for (int i = 0; i < 4; i++) glVertex2f(v[i][0], v[i][1]);
    glEnd();
    glColor3f(br, bg, bb);
    glLineWidth(lw);
    glBegin(GL_LINE_LOOP);
    for (int i = 0; i < 4; i++) glVertex2f(v[i][0], v[i][1]);
    glEnd();
}

void drawString(float x, float y, const char* s) {
    glRasterPos2f(x, y);
    for (; *s; s++) glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, *s);
}

void drawAxes() {
```

```
    glColor3f(0.8f,0.8f,0.8f);
    glLineWidth(1.5f);
    glBegin(GL_LINES);
    glVertex2f(-8,0); glVertex2f(8,0);
    glVertex2f(0,-2); glVertex2f(0,10);
    glEnd();
    glBegin(GL_TRIANGLES);
    glVertex2f(7.8f,0.15f); glVertex2f(8,0); glVertex2f(7.8f,-0.15f);
    glVertex2f(-0.15f,9.8f); glVertex2f(0,10); glVertex2f(0.15f,9.8f);
    glEnd();
    char buf[10];
    for (int x = -7; x <= 7; x++) {
        if (x == 0) continue;
        glBegin(GL_LINES); glVertex2f(x,-0.1f); glVertex2f(x,0.1f); glEnd();
        sprintf(buf,"%d",x); drawString(x-0.2f,-0.4f,buf);
    }
    for (int y = -1; y <= 9; y++) {
        if (y == 0) continue;
        glBegin(GL_LINES); glVertex2f(-0.1f,y); glVertex2f(0.1f,y); glEnd();
        sprintf(buf,"%d",y); drawString(0.15f,y-0.15f,buf);
    }
    glBegin(GL_LINES);
    glVertex2f(-0.1f,0); glVertex2f(0.1f,0);
    glVertex2f(0,-0.1f); glVertex2f(0,0.1f);
    glEnd();
    drawString(-0.3f,-0.4f,"0");
    drawString(7.6f,-0.5f,"X");
    drawString(0.15f,9.6f,"Y");
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();
    // original (dashed, optional)
    glEnable(GL_LINE_STIPPLE);
    glLineStipple(1,0x00FF);
    glColor3f(0.5f,0.5f,0.5f);
    glBegin(GL_LINE_LOOP);
    for (int i=0;i<4;i++) glVertex2f(original[i][0],original[i][1]);
    glEnd();
    glDisable(GL_LINE_STIPPLE);
    // Translated figure: GREEN border + CREAM fill
```

```
    drawPolygon(translated, 1.0f,0.992f,0.816f, 0.0f,1.0f,0.0f, 2.5f);
    // Rotated figure (any style, here light peach + red)
    drawPolygon(rotated, 1.0f,0.9f,0.8f, 1.0f,0.0f,0.0f, 2.0f);
    glutSwapBuffers();
}

void reshape(int w, int h) {
    glViewport(0,0,w,h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-8,8,-2,10);
    glMatrixMode(GL_MODELVIEW);
}

void printCoords() {
    printf("Original: A(0,4) B(3,4) C(4,0) D(0,0)\n");
    printf("Translated: A'(%.2f,%.2f) B'(%.2f,%.2f) C'(%.2f,%.2f) D'(%.2f,%.2f)\n",
           translated[0][0],translated[0][1],translated[1][0],translated[1][1],
           translated[2][0],translated[2][1],translated[3][0],translated[3][1]);
    printf("Rotated (55°): A''(%.2f,%.2f) B''(%.2f,%.2f) C''(%.2f,%.2f)
D''(%.2f,%.2f)\n",
           rotated[0][0],rotated[0][1],rotated[1][0],rotated[1][1],
           rotated[2][0],rotated[2][1],rotated[3][0],rotated[3][1]);
}

int main(int argc, char** argv) {
    theta_rad = theta_deg * PI/180.0;
    computeTranslated();
    computeRotated();
    printCoords();
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800,600);
    glutCreateWindow("OpenGL: Translation (green/cream) + Rotation (55deg)");
    glClearColor(0.1f,0.1f,0.15f,1.0f);
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();
    return 0;
}
```

Complete Python Version (copy-paste into `quadrilateral.py`)

```python
#!/usr/bin/env python3
"""
Group 11 – Eunice Nyaboke & Victor Ireri
OpenGL: Translation (green/cream) + Rotation (55°) with labelled axes
"""

import sys
import math
from OpenGL.GL import *
from OpenGL.GLU import *
from OpenGL.GLUT import *

original = [[0,4],[3,4],[4,0],[0,0]]
translated = [[0,0] for _ in range(4)]
rotated = [[0,0] for _ in range(4)]
tx, ty = 2, 2
theta_deg = 55
theta_rad = math.radians(theta_deg)

def compute_translated():
    for i in range(4):
        translated[i][0] = original[i][0] + tx
        translated[i][1] = original[i][1] + ty

def compute_rotated():
    c = math.cos(theta_rad)
    s = math.sin(theta_rad)
    for i in range(4):
        x, y = translated[i]
        rotated[i][0] = x*c - y*s
        rotated[i][1] = x*s + y*c

def draw_polygon(v, fill, border, lw=2.5):
    glColor3f(*fill)
    glBegin(GL_POLYGON)
    for p in v: glVertex2f(p[0], p[1])
```

```
    glEnd()
    glColor3f(*border)
    glLineWidth(lw)
    glBegin(GL_LINE_LOOP)
    for p in v: glVertex2f(p[0], p[1])
    glEnd()

def draw_string(x, y, text):
    glRasterPos2f(x, y)
    for ch in text:
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, ord(ch))

def draw_axes():
    glColor3f(0.8,0.8,0.8)
    glLineWidth(1.5)
    glBegin(GL_LINES)
    glVertex2f(-8,0); glVertex2f(8,0)
    glVertex2f(0,-2); glVertex2f(0,10)
    glEnd()
    glBegin(GL_TRIANGLES)
    glVertex2f(7.8,0.15); glVertex2f(8,0); glVertex2f(7.8,-0.15)
    glVertex2f(-0.15,9.8); glVertex2f(0,10); glVertex2f(0.15,9.8)
    glEnd()
    for x in range(-7,8):
        if x==0: continue
        glBegin(GL_LINES); glVertex2f(x,-0.1); glVertex2f(x,0.1); glEnd()
        draw_string(x-0.2,-0.4,str(x))
    for y in range(-1,10):
        if y==0: continue
        glBegin(GL_LINES); glVertex2f(-0.1,y); glVertex2f(0.1,y); glEnd()
        draw_string(0.15,y-0.15,str(y))
    glBegin(GL_LINES)
    glVertex2f(-0.1,0); glVertex2f(0.1,0)
    glVertex2f(0,-0.1); glVertex2f(0,0.1)
    glEnd()
    draw_string(-0.3,-0.4,"0")
    draw_string(7.6,-0.5,"X")
    draw_string(0.15,9.6,"Y")

def display():
    glClear(GL_COLOR_BUFFER_BIT)
```

```
    draw_axes()
    # original (dashed, optional)
    glEnable(GL_LINE_STIPPLE)
    glLineStipple(1, 0x00FF)
    glColor3f(0.5,0.5,0.5)
    glBegin(GL_LINE_LOOP)
    for p in original: glVertex2f(p[0], p[1])
    glEnd()
    glDisable(GL_LINE_STIPPLE)
    # Translated figure: GREEN border + CREAM fill
    draw_polygon(translated, (1.0,0.992,0.816), (0.0,1.0,0.0), 2.5)
    # Rotated figure (light peach + red)
    draw_polygon(rotated, (1.0,0.9,0.8), (1.0,0.0,0.0), 2.0)
    glutSwapBuffers()

def reshape(w, h):
    glViewport(0,0,w,h)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(-8,8,-2,10)
    glMatrixMode(GL_MODELVIEW)

def print_coords():
    print("Original: A(0,4) B(3,4) C(4,0) D(0,0)")
    print(f"Translated: A'({translated[0][0]:.2f},{translated[0][1]:.2f})
B'({translated[1][0]:.2f},{translated[1][1]:.2f}) "
          f"C'({translated[2][0]:.2f},{translated[2][1]:.2f})
D'({translated[3][0]:.2f},{translated[3][1]:.2f})")
    print(f"Rotated (55°): A''({rotated[0][0]:.2f},{rotated[0][1]:.2f})
B''({rotated[1][0]:.2f},{rotated[1][1]:.2f}) "
          f"C''({rotated[2][0]:.2f},{rotated[2][1]:.2f})
D''({rotated[3][0]:.2f},{rotated[3][1]:.2f})")

def main():
    compute_translated()
    compute_rotated()
    print_coords()
    glutInit(sys.argv)
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB)
    glutInitWindowSize(800,600)
    # Use bytes literal without degree symbol to avoid Windows error
```

```
    glutCreateWindow(b"OpenGL: Translation (green/cream) + Rotation (55 deg)")
    glClearColor(0.1,0.1,0.15,1.0)
    glutDisplayFunc(display)
    glutReshapeFunc(reshape)
    glutMainLoop()

if __name__ == "__main__":
    main()
```

How to Run

C++ (Linux / macOS / Windows with freeglut)

```bash
g++ main.cpp -o opengl_demo -lGL -lGLU -lglut
./opengl_demo
```

Python (Windows / Linux / macOS)

```bash
pip install PyOpenGL PyOpenGL_accelerate
python quadrilateral.py
```

Note for Windows Python: The window title uses a **bytes literal** (`b"..."`) without the degree symbol to avoid `ctypes.ArgumentError`.

Expected Output

- Console prints original, translated, and rotated coordinates.
- Window shows:
  - Light grey labelled axes with ticks.
  - Dashed outline of original quadrilateral.
  - **Translated quadrilateral** with **cream fill** and **thick green border**.
  - Rotated quadrilateral (light peach fill + red border) for verification.